Toward the Photorealistic Rendering of Study Room Scenes through Monte Carlo Path Tracing with a Customised Lightweight BSDF and Disney BSDF

Yuan Hong
Australian National University
Acton, ACT 2601
u8010795@anu.edu.au

Haoting Chen
Australian National University
Acton, ACT 2601
u7227871@anu.edu.au

Hongyue Yu
Australian National University
Acton, ACT 2601
u8068619@anu.edu.au

Abstract

This paper presents a C++ photorealistic rendering pipeline based on Monte Carlo Path Tracing, featuring both a lightweight custom BSDF and the Disney BSDF to simulate diverse lighting effects and materials. Built solely on standard C++ libraries, the pipeline illustrates key physical and probabilistic concepts, making it valuable for educational purposes. We document typical bugs and physical phenomena encountered during development to support future learners. Evaluated on a study room scene, the pipeline effectively simulates diffuse and specular reflection, absorption, and common materials like wood and metal. Future extensions could enhance realism, including hair and gas rendering.

1 Introduction

Photorealistic rendering aims to generate images from 3D scenes with lighting and material effects that closely resemble the real world. This technique is useful for applications like interior design and product visualization. While existing tools such as Blender support photorealistic rendering via ray tracing or rasterization, building a simple rendering pipeline from scratch provides valuable insight into the underlying physics and probabilistic models, benefiting learners in computer graphics.

This project was inspired by a competition-winning image (Figure 1), which showcases some realistic effects like specular and diffuse reflection on the iMac screen and soft shadows. However, it lacks features such as window refraction, transparent glass, and accurate shadowing. We use this image as a baseline to test and evaluate our pipeline, aiming to replicate its effects and enhance realism further. To support this, we constructed a similar, more complex study room scene for direct visual comparison.

The remaining part of this paper is structured as follows. Section 2 provides the necessary background for Ray Tracing, Monte Carlo Path Tracing, Next Event Estimation, and BSDFs. Section 3 outline the methodology we used to design our render pipeline. Section 4 evaluate our pipeline based on both efficiency and overall image rendering quality, as well as highlight the typical bugs and interesting physical phenomena during the development to facilitate the development of similar projects in the future. Section 5 concludes the paper and provides direction for future work.

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

Figure 1: The Motivation Image

2 Background

Ray Tracing refers to a broad class of rendering technology that traces rays backward from the camera, image plane, objects they reflect from or transmit through, and ultimately back to the light source to determine the colours of pixels on the image plane based on the intensity of rays when they hit the image plane.

2.1 Physically Based Ray Tracing

Physically Based Ray Tracing (PBRT) simulates light transport using physically accurate models, adhering to the Law of Conservation of Energy. It typically relies on the Rendering Equation and BSDFs to describe how light reflects and refracts on surfaces, enabling realistic effects like mirror reflections, soft shadows, and glass refractions.

Like traditional Whitted-Style Ray Tracing, PBRT traces rays from the image plane into the scene. However, it computes the radiance—the light's power per unit area per solid angle—of each pixel, typically a 3D vector, to determine the RGB colour values of a pixel. PBRT also considers many or all objects, instead of just the last object a ray hits.

When a ray hits a surface, its outgoing radiance depends on incoming light from the upper hemisphere (and the lower hemisphere if the material is transmissive). Calculating this involves recursively tracing and integrating the radiance of incoming rays, stopping when a ray hits a light source or reaches a maximum depth. This recursive process ensures all light energy at the image plane originates from actual light sources, preserving energy conservation. Assume an object has no emission or absorption, then the outgoing radiance in a direction follows the Rendering Equation.

$$L_o(\mathbf{x}, \omega_o) = \int_{\Omega_+} f_r(\mathbf{x}, \omega_i, \omega_o) L_i(\mathbf{x}, \omega_i)(\omega_i \cdot \mathbf{n})\, d\omega_i + \int_{\Omega_-} f_t(\mathbf{x}, \omega_i, \omega_o) L_i(\mathbf{x}, \omega_i)|\omega_i \cdot \mathbf{n}|\, d\omega_i$$

fr and ft are the BSDF functions that determine the reflection and transmission properties of a material, that is, how outgoing rays are distributed based on incoming rays. $\omega_i \cdot \mathbf{n}$ accounts for the angular attenuation of light. L_i is the incoming radiance of a light. For a perfect glass, the outgoing rays are determined by only two incoming rays, one for reflection and the other for refraction, leading to a branching factor of two, which is complex but still possible to be solved by a computer. However, for a Lambertian surface, the outgoing rays are determined by an infinite number of incoming rays around the top hemisphere on the surface, making solving this integral by brute force or analytically impossible.

2.2 Monte Carlo Path Tracing

Therefore, we implement Monte Carlo Path Tracing, which uses only one sampled incoming ray to determine each outgoing ray, but later divides the result by the probability of choosing this ray so that

we do not underestimate the contributions from all incoming rays to the outgoing rays. This led to linear recursion when tracing each ray backward. The Monte Carlo Approximation for an integral is as follows.

$$\int_{\Omega} g(\omega)\, d\omega \approx \frac{1}{N} \sum_{i=1}^{N} \frac{g(\omega_i)}{p(\omega_i)}$$

$g(w_i)$ is what is inside the integral in the rendering function, and $p(\omega_i)$ is the sampling function. The sampling function is usually chosen to be close to $g(w_i)$ to increase convergence speed. However, the Monte Carlo Integral Approximation remains unbiased if the sampling function gives all values a chance. To ensure the rendered image is close to the ground truth, we typically sample and trace many rays hitting a pixel, i.e., increase N. Each ray will remain a single ray through the recursion. However, the specific path it takes depends on the sampling.

2.3 Next Event Estimation

$$L_i(\mathbf{x},\, \omega_o) = L_{\text{direct}} + L_{\text{indirect}}$$

$$L_{\text{direct}} = \frac{f_s(\omega_l, \omega_o)\;\; L_e\big(\mathbf{x}_l,\, -\omega_l\big)\;\; \big(\mathbf{n}_x \cdot \omega_l\big)\;\; \big(\mathbf{n}_l \cdot (-\omega_l)\big)\;\; \text{isVisiable}(\mathbf{x}, \mathbf{x}_l)}{\|\mathbf{x} - \mathbf{x}_l\|^2\; p_{\text{light}}(\mathbf{x}_l)}$$

$$L_{\text{indirect}} = \frac{f_s(\omega_i, \omega_o)\;\; L_o\big(\mathbf{x}',\, -\omega_i\big)\;\; \big|\mathbf{n}_x \cdot \omega_i\big|}{p_{\text{bsdf}}(\omega_i)}$$

However, pure Monte Carlo Path Tracing has a low convergence speed, i.e., requires N a very large number or the Russian Roulette to be very close to 1 (i.e., the average $MaxDepth$ is very high) to be close to the ground truth. Unfortunately, most light rays may never hit the light source to obtain a colour within a limited number of depths or be lost in empty space, leading to a strong noise, especially salt and pepper noise when N and $MaxDepth$ is small. Next Event Estimation (NEE) can be considered a conditional bonus to each light ray, but it still ensures that the overall result is unbiased. The original rendering equation is split into two parts, radiance from direct and indirect lighting. When a ray hits an object that can be directly illuminated by a sampled light source, the radiance from the light source to the object is immediately returned. However, since this ray bounces from direct light once, when it goes deeper into the recursion of the indirect lighting part and eventually hits the light source, no radiance is returned. In contrast, if a light ray always travels under the shadow of objects, i.e., L_{direct} is always zero in at each recursion depth, but eventually hits the light source, it can still obtain the radiance from the light source.

2.4 BSDFs

Light exhibits different reflection and transmission behaviour when hitting or travelling through different materials. This is why materials such as wood, metal, and fabric look different in the real world. To achieve photorealistic rendering, we typically need different BSDFs for different materials. However, real-world materials are complex, and we sometimes idealise the reflection and transmission properties of a material to some extent or mimic a material with another material with totally different physical properties, but that looks the same. Even so, a good modelling of material using a BSDF can lead to quite a photorealistic rendering result. Even if the material model is inaccurate, the Monte Carlo path tracing process can still be physically correct and energy-conserving. However, such a material may not exist in the real world.

3 Methodology

To continuously develop and evaluate our Monte Carlo Path Tracing rendering pipeline, we have divided the process into three subtasks: “modelling and scene building”, “pipeline structure and main ray tracing logic design”, and “BSDF implementation for various materials”. Each seems to be built on top of the other, but in fact, they can be done in parallel and correspond to the main task of each member in our team.

3.1 Modelling and Scene Building

Modelling and scene building form the test set for our pipeline. The indoor study scene must be sufficiently complex to exercise every feature we've implemented for demonstration and evaluation. Accordingly, we've arranged a variety of items in the study room, each with distinct material properties:

(1) A body mirror to test perfect specular reflection on fully opaque objects. (2) An iMac screen to evaluate the combination of specular and diffuse reflection on fully opaque surfaces. (3) A window and a glass bunny to examine light behavior through perfect glass (fully transmissive objects). (4) A semi-transparent guitar and a water glass to test combined specular reflection, refraction, and diffuse reflection on semi-transparent materials. (5) A human character to verify that the pipeline can handle non-manifold geometry, since the model's hair is constructed from planes rather than volumetric meshes. (6) Two light sources—one ceiling fixture and one desk lamp—to test multiple-illumination scenarios. (7) A floor with an implicitly defined pattern to ensure correct handling of both explicitly mapped textures and procedural mapping. (8) Objects with various materials—a wooden table, a metal flute, and the character's clothing—to validate our rendering pipeline across different BSDFs.

We use Blender for scene building. Selected 3D models—often not in OBJ format—are imported into Blender, cleaned up, transformed into position, and assigned appropriate BSDF parameters (e.g., kd, ks, roughness, metalness). Camera placement and field of view are also configured in Blender. Once the scene is finalized, it is exported as one or more triangulated OBJ files. On the programming side, the only modification required is loading the updated OBJ files and camera data, ensuring a seamless transition from Task 1 to Task 2. Both the scene and the pipeline evolve incrementally to support and test an expanding set of features. We choose Blender over our ray tracing pipeline for modelling because its rasterization renders far more quickly, enabling efficient object adjustments. The sole exception is light setup, which must be configured directly in the ray tracer due to its fundamentally different lighting model.

3.2 Pipeline Structure and Main Ray Tracing Logic Design

Task 2 involves defining a good pipeline structure, writing the main Monte Carlo Path Tracing logic, and implementing a lightweight BSDF to support a limited number of materials, whereas Task 3 extend this BSDF to the more advanced Disney BSDF. We chose to build our pipeline on top of the Assignment 4 framework of the Computer Graphics course of the Australian National University (ANU) since it contains almost all the necessary logic for loading OBJ, locating the camera, shooting rays from the image plane, and gathering results to generate images. The key part to achieving Monte Carlo Path Tracing is to implement a "cast_ray" function that controls how a ray shot from each pixel interacts with the scene so that its colour is known.

Figure 2 shows the main logic of Monte Carlo Path Tracing with our customised lightweight BSDF. Every block coloured purple is a Monte Carlo Approximation of an integral, that is, a ray only chooses one path to go, but its radiance is divided by the product of all the probabilities that lead it to the current node so far. A ray first goes through Russian Roulette, a special case of the Monte Carlo Approximation of an integral with only two outcomes/values. This determines the average max depth at which we trace the ray. For example, if we choose 0.9, it will lead to an average of $1/(1-0.9) = 10$ max depth. If a ray survives from Russian Roulette, we also need to ensure that a ray is from something, either a light source or an object. Otherwise, it returns 0. If the ray is from a light source, whether the radiance of the light source returns depends on whether the ray has obtained the NEE bonus during its outer recursion. This ensures the Law of Conservation of Energy, i.e., we do not count the radiance of a light source twice. After the light check, the remaining part of "cast_ray" is all about implementing the rendering equation with a customised lightweight BSDF. Although we did not explicitly have a function called BSDF, the logic plus the Lambertian Bidirectional Reflection Distribution Function (BRDF) used in the helper function "compute_lambertian_diffuse_reflection" collectively form a BSDF.

In our initial lightweight BSDF, a material is defined only by five properties, ks, kd, ρ, d, and ior. These properties also affect how an incoming ray is sampled and how the radiance of the outgoing ray is computed from the incoming ray. kd is the diffuse coefficient for a fully opaque object, telling how much light performs Lambertian diffuse reflection when hitting the object. ks is the specular coefficient for a fully opaque object, telling how much light performs perfect specular reflection.

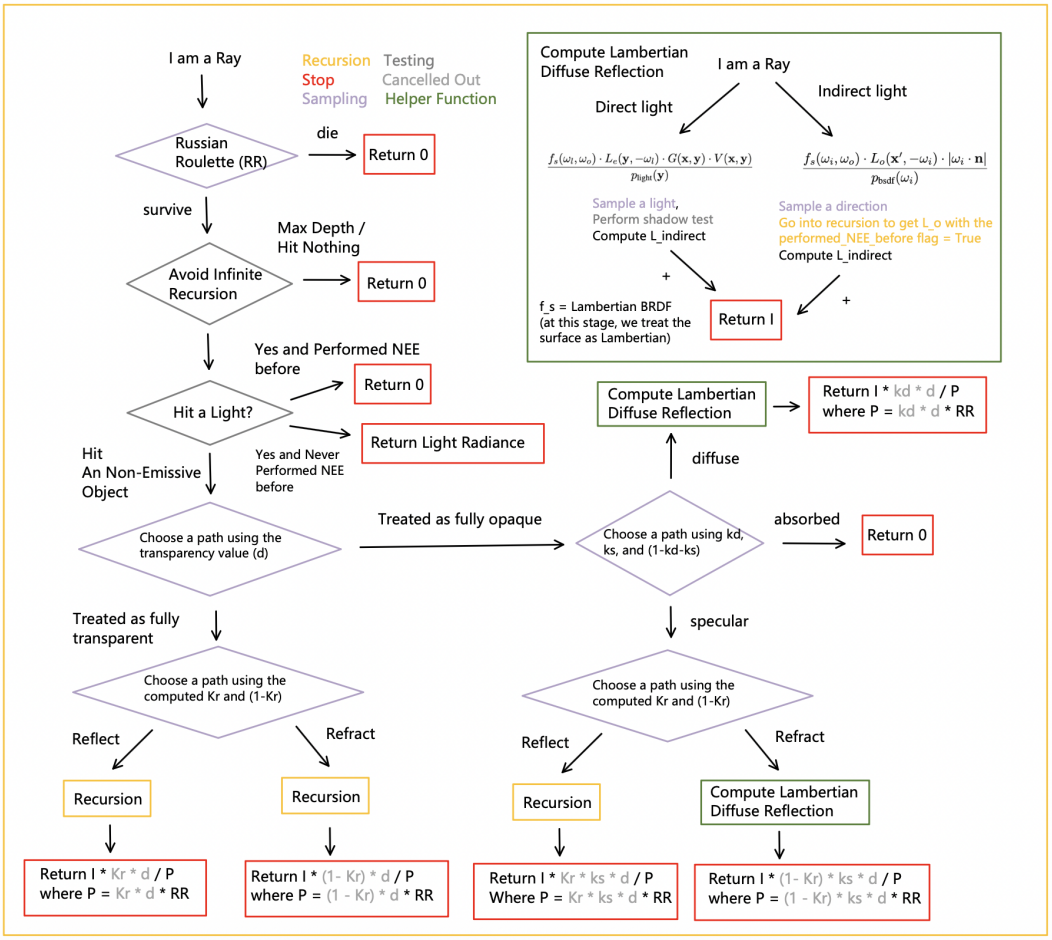

Figure 2: Main Ray Tracing Logic + Our Customised Lightweight BSDF

Hence, $1 - kd - ks$ is then the proportion of light that is absorbed by the object. These three coefficients (kd, ks, $1 - kd - ks$) exhaustively partition how light interacts with a fully-opaque surface in physics. ρ is used by Lambertian BRDF to control the base colour of an object. ior affects the proportion of reflection and refraction for a fully transparent object or the proportion of reflection and subsurface diffuse reflection for a fully opaque object. d controls the transparency of the material, that is, how much a material is made up of fully opaque material and fully transparent material. While the use of d allows us to easily mimic a semi-transparent material, such as frosted glass, transparency is not a rigorous physics term. An object in the real world that looks semi-transparent is not because it is a combination of a fully transparent and a fully opaque object.

When the inverse outgoing ray hits the object, based on sampling d, we either treat it as fully opaque or fully transparent. If it is treated as fully transparent, we continue to randomly treat the object that can only perform reflection or refraction. If it is treated as fully opaque, we randomly choose to perform diffuse reflection, specular reflection, or absorption. If diffuse reflection is chosen, an incoming ray direction is sampled to compute the part of radiance of the outgoing light contributed by the incoming light, and a point on the light is sampled to compute the part of radiance of the outgoing light contributed by direct light. As long as NEE is performed, i.e., the ray gets a bonus from direct light, we enable the "perform_NEE_before" flag for later recursion to ensure the light is only counted once.

One major adjustment apart from the "cast_ray" function to the Assignment 4 framework is that all the properties of a material, ks, kd, ρ, d, and ior, are no longer scalers of 3D vectors. They are defined by a float or Vector3 matrix or a Vector2 to float or Vector3 function. This allows the texture mapping to not only the base colour rho, but also all the other properties, which allow a mesh to have

different material properties at different surface points. The support for implicit texture mapping through a function also enables the definition of mathematical patterns without the need to use an image.

Given the lightweight BSDF, we expect to simulate some materials, such as plastic ($ks = 0.2$, $kd = 0.8$, $ior = 1.46$, $d = 1$), a perfect mirror ($ks = 1$, $kd = 0$, $ior = \infty$, $d = 1$), coloured mirror or the closed iMac screen ($ks = 0.8$, $kd = 0.2$, $ior = 3$, $d = 1$), a wall ($ks = 0.04$, $kd = 0.96$, $ior = 1.376$, $d = 1$), human skin ($ks = 0.1$, $kd = 0.9$, $ior = 1.5$, $d = 1$), fully transparent glass ($ks = kd = undefined$, $ior = 1.5$, $d = 0$), and semi-transparent glass ($ks = 0$, $kd = 1$, $ior = 1.5$, $d = 0.1$).

However, the current lightweight BSDF struggles to simulate some complex materials, which limits the photorealistic rendering for our study room scene. In addition, the current BSDF is not a standard way of defining a material. A large portion of 3D models nowadays are defined through a more advanced BSDF, such as Disney BSDF, leading to an extra conversion from standard BSDF parameters to our lightweight BSDF parameters. Also, introducing the micro-surface model makes sampling more realistic. Hence, we tried another pipeline , the Disney BSDF.

3.3 Disney BSDF Implementation for Various Materials

Due to the limitation of the current Assignment 4 framework of ANU, which only defines a material with a limited number of properties, we have also switched our framework to the Assignment 1 framework of the Advanced Image Synthesis course of the University of California, San Diego (UCSD). We have implemented all the key functions for Monte Carlo Path Tracing and BSDF materials except some helper functions and scene manager provided as part of the framework.

To support the need for rendering a wider variety of materials with greater visual realism, we adopt a microfacet-based model and implement the Disney BSDF . This model enables more physically plausible handling of phenomena such as subsurface scattering and refraction, ensuring better energy conservation.

The Disney BSDF introduces 12 unified parameters and categorizes materials into five major classes: opaque dielectrics, transmissive dielectrics, metals, clearcoat (a secondary specular layer), and sheen (retroreflective effects for cloth and fibers). In the following sections, we describe each component in detail.

- **BaseColor**: The intrinsic surface color, also referred to as albedo.
- **Subsurface**: Controls the degree of subsurface scattering, used for materials like skin or marble.
- **SpecularTransmission**: The fraction of energy transmitted through the surface (for transparent dielectrics).
- **Specular**: The intensity of specular reflection for non-metallic surfaces.
- **Metallic**: Indicates the material's metallic nature, interpolating between dielectric and conductor behaviors.
- **Roughness**: Controls surface micro-roughness, affecting the sharpness of reflections.
- **SpecularTint**: Modulates the tint of the specular reflection between white and `BaseColor`.
- **Anisotropic**: Specifies the degree of anisotropy in the surface, affecting highlight shape.
- **Sheen**: Adds an extra grazing-angle reflectance, simulating cloth-like materials.
- **SheenTint**: Adjusts the color of the sheen lobe from white to `BaseColor`.
- **Clearcoat**: Introduces a secondary specular lobe on top of the surface, useful for varnished materials.
- **ClearcoatGloss**: Controls the roughness of the clearcoat layer, affecting its sharpness.

The total light distribution is in Figure 3

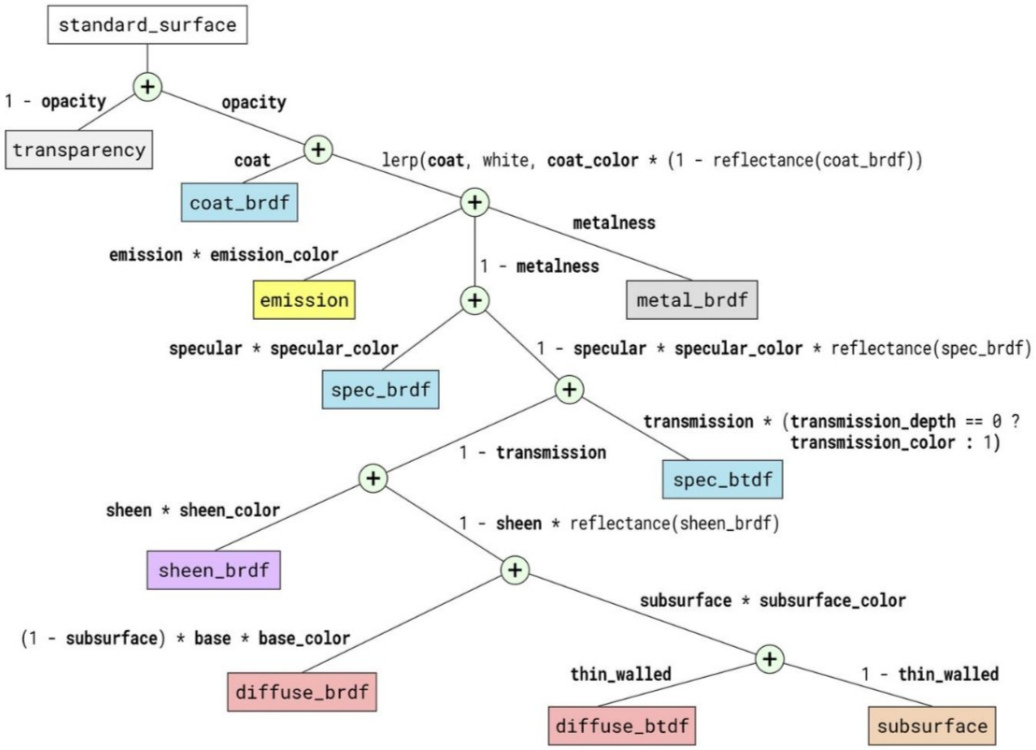

Figure 3: Disney BSDF Light Distribution

3.3.1 Diffuse (BaseColor + Subsurface)

The diffuse component blends a Lambertian-like base term and an approximate subsurface scattering model:

$$f_{\text{diffuse}} = (1 - \text{subsurface}) \cdot f_{\text{base}} + \text{subsurface} \cdot f_{\text{subsurface}}$$

$$f_{\text{base}} = \frac{\text{baseColor}}{\pi} F_D(\omega_{in}) F_D(\omega_{out})$$

$$F_D(\omega) = 1 + (F_{D90} - 1)(1 - |\mathbf{n} \cdot \omega|)^5, \quad F_{D90} = \frac{1}{2} + 2 \cdot \text{roughness} \cdot (\mathbf{h} \cdot \omega)^2$$

$$f_{\text{subsurface}} = \frac{1.25 \cdot \text{baseColor}}{\pi} \left[F_{SS}(\omega_{in}) F_{SS}(\omega_{out}) \left(\frac{1}{|\mathbf{n} \cdot \omega_{in}| + |\mathbf{n} \cdot \omega_{out}|} - 0.5 \right) + 0.5 \right]$$

The original Schlick approximation models the angle-dependent Fresnel reflection for specular highlights:

$$F(\omega) = R_0 + (1 - R_0)(1 - \cos\theta)^5 \tag{1}$$

where $R_0 = \left(\frac{n_1 - n_2}{n_1 + n_2}\right)^2$ is the reflectance at normal incidence, and θ is the angle between the incident/viewing direction and the surface normal.

To simulate increased reflectance at grazing angles in diffuse materials, Disney's model uses the following modified Fresnel function:

$$F_D(\omega) = 1 + (F_{D90} - 1)(1 - |\mathbf{n} \cdot \omega|)^5 \tag{2}$$

- F_{D90} is a grazing-angle boost factor, defined as:

$$F_{D90} = 1 + 2 \cdot \text{roughness} \cdot (\mathbf{h} \cdot \omega)^2 \tag{3}$$

- $\mathbf{h} = \frac{\omega_i + \omega_o}{\|\omega_i + \omega_o\|}$ is the half-vector between light and view directions

As the angle between ω and the surface normal $\mathbf{n}$ increases, $|\mathbf{n} \cdot \omega| \to 0$, and thus:

$$(1 - |\mathbf{n} \cdot \omega|)^5 \to 1 \quad \Rightarrow \quad F_D(\omega) \to F_{D90} \tag{4}$$

This implies that at grazing angles, the diffuse reflectance is dominated by F_{D90}.

Effect of $(\mathbf{h} \cdot \omega)^2$ and Roughness

- When $\omega \approx \mathbf{h}$ (i.e., when the light and view directions are similar), then $(\mathbf{h} \cdot \omega)^2 \to 1$, and F_{D90} becomes large. This simulates the *retro-reflection* effect.
- When $\mathbf{h} \cdot \omega \to 0$, the grazing effect is reduced.
- The **roughness** parameter controls the strength of this effect:
  - Higher roughness ⇒ stronger grazing-angle boost
  - Lower roughness ⇒ behavior closer to Lambertian diffuse

Cosine-Weighted Hemisphere Sampling
In Monte Carlo integration for physically based rendering, it is common to employ *cosine-weighted hemisphere sampling*, where directions are drawn according to the probability density function (PDF)

$$\text{PDF}(\omega) = \frac{\cos\theta}{\pi},$$

with θ denoting the angle between the sampled direction ω and the surface normal $\mathbf{n}$. This sampling strategy is motivated by the $\cos\theta$ term in the rendering equation, which assigns greater importance to directions aligned with the surface normal. By matching the sampling distribution to this term, the variance of the Monte Carlo estimator is significantly reduced, resulting in more efficient sampling and faster convergence in path tracing algorithms.

3.3.2 Metal.

Metallic reflection is modeled using the anisotropic Cook-Torrance microfacet BRDF:

$$f_{\text{metal}} = \frac{F_m D_m G_m}{4|\mathbf{n} \cdot \omega_{in}|}, \quad F_m = \text{baseColor} + (1 - \text{baseColor})(1 - \mathbf{h} \cdot \omega_{out})^5$$

$$D_m = \frac{1}{\pi \alpha_x \alpha_y} \left(\frac{1}{(\frac{h_x^2}{\alpha_x^2} + \frac{h_y^2}{\alpha_y^2} + h_z^2)^2} \right)$$

$$G(\omega) = \frac{1}{1 + \Lambda(\omega)}, \quad \Lambda(\omega) = \frac{\sqrt{1 + \frac{(\omega_x \alpha_x)^2 + (\omega_y \alpha_y)^2}{\omega_z^2}} - 1}{2}, \quad G_m = G(\omega_{in}) G(\omega_{out})$$

where:

- ω_i is the incoming light direction,
- ω_o is the outgoing view direction,
- $\mathbf{n}$ is the surface normal,
- $\omega_h = \frac{\omega_i + \omega_o}{\|\omega_i + \omega_o\|}$ is the half-vector between light and view directions.

This formulation includes three physically meaningful components:

- $F(\omega_h)$: the **Fresnel term**, describing the proportion of light that reflects off a surface at a given angle.
- $D(\omega_h)$: the **normal distribution function** (NDF), representing the distribution of microfacet orientations around the half-vector.
- $G(\omega_i, \omega_o)$: the **geometry term**, accounting for masking and shadowing due to microfacet occlusion.

VNDF (visible normal distribution function) sampling employs the probability density function

$$\text{PDF} = \frac{D_m \cdot G(\omega_{in})}{4|\omega_{in} \cdot \mathbf{n}|},$$

where sampling is restricted to the set of microfacets visible from the outgoing direction. This approach implicitly incorporates the geometry term $G_1(\omega_o)$, accounting for masking effects. Compared to sampling the full normal distribution function (NDF), VNDF sampling better aligns with the energy-contributing directions and reduces variance in Monte Carlo integration, particularly for rough surfaces where many microfacets are self-occluded.

3.3.3 Glass.

A combination of reflection and refraction:

$$f_{\text{glass}} = \begin{cases} \dfrac{\text{baseColor} \cdot F_g \cdot D_g \cdot G_g}{4|\mathbf{n} \cdot \omega_{\text{in}}|} & \text{if } (\mathbf{n}_g \cdot \omega_{\text{in}})(\mathbf{n}_g \cdot \omega_{\text{out}}) > 0 \\ \dfrac{\sqrt{\text{baseColor}}(1 - F_g) D_g G_g \, |\mathbf{h} \cdot \omega_{\text{out}}| \, |\mathbf{h} \cdot \omega_{\text{in}}|}{|\mathbf{n} \cdot \omega_{\text{in}}| \, |\mathbf{h} \cdot \omega_{\text{in}} + \eta \, \mathbf{h} \cdot \omega_{\text{out}}|^2} & \text{otherwise} \end{cases} \tag{5}$$

Fresnel term F_g uses true Fresnel equations:

$$R_s = \frac{\mathbf{h} \cdot \omega_{in} - \eta \mathbf{h} \cdot \omega_{out}}{\mathbf{h} \cdot \omega_{in} + \eta \mathbf{h} \cdot \omega_{out}}, \quad R_p = \frac{\eta \mathbf{h} \cdot \omega_{in} - \mathbf{h} \cdot \omega_{out}}{\eta \mathbf{h} \cdot \omega_{in} + \mathbf{h} \cdot \omega_{out}}, \quad F_g = \frac{1}{2}(R_s^2 + R_p^2)$$

Real Fresnel is preferred because the Schlick approximation becomes inaccurate when the IOR nears 1.

The term $\sqrt{\text{baseColor}}$ in the transmission branch approximates the color attenuation that occurs as light travels through the medium. This accounts for energy loss due to absorption along the refracted path inside the material.

The denominator term $|\mathbf{h} \cdot \omega_{\text{in}} + \eta \, \mathbf{h} \cdot \omega_{\text{out}}|^2$ models the energy loss caused by internal transmission distortion. Specifically, it represents how the refracted direction deviates from the ideal transmission path due to the index of refraction η. As light bends through the surface, some energy is dispersed or absorbed by the microstructure of the medium, and this denominator helps to attenuate the BSDF accordingly.

3.3.4 ClearCoat.

Introduced to simulate longer highlight tails not captured by metal (GGX/GTR2). ClearCoat uses GTR1 and adds a second specular layer:

$$f_{\text{clearcoat}} = \frac{F_c D_c G_c}{4|\mathbf{n} \cdot \omega_{in}|}$$

$$F_c = R_0 + (1 - R_0)(1 - \mathbf{h} \cdot \omega_{out})^5, \quad R_0 = \left(\frac{1.5 - 1}{1.5 + 1}\right)^2$$

GTR(Generalized Trowbridge-Reitz)

$$D_{\text{GTR}} = \frac{c}{\left(\alpha^2 \cos^2 \theta_h + \sin^2 \theta_h\right)^{\gamma}} \tag{6}$$

the parameter γ controls the falloff rate of the specular highlight. Specifically, a larger γ leads to a faster decay of the distribution away from the highlight direction, resulting in sharper and narrower specular lobes.

The Schlick Fresnel term F_c has a hard-coded index of refraction $\eta = 1.5$. The normal distribution function D_c uses an isotropic roughness $\alpha = \alpha_g$. The masking-shadowing term G_c uses a fixed roughness value of 0.25. Note that this is an *ad-hoc* fit, and there is no clear geometric meaning for this microfacet BRDF.

Sampling is based on D_c. PDF: $\frac{D_c|\mathbf{n} \cdot \mathbf{h}|}{4|\mathbf{h} \cdot \omega_{in}|}$.

3.3.5 Sheen.

Designed for cloth-like surfaces with grazing retro-reflection:

$$f_{\text{sheen}} = C_{\text{sheen}}(1 - |\mathbf{h} \cdot \omega_{out}|)^5 |\mathbf{n} \cdot \omega_{out}|,$$
$$C_{\text{sheen}} = (1 - \text{sheenTint}) + \text{sheenTint} \cdot \text{baseColor}/\text{luminance}(\text{baseColor})$$

Sampling and PDF match cosine hemisphere logic.

3.3.6 Disney Principled BSDF.

The full BSDF is:

$$\begin{aligned} f_{\text{Disney}} = &(1 - \text{specTrans})(1 - \text{metallic}) f_{\text{diffuse}} + \\ &(1 - \text{metallic}) \cdot \text{sheen} \cdot f_{\text{sheen}} + \\ &(1 - \text{specTrans}(1 - \text{metallic})) \cdot \hat{f}_{\text{metal}} + \\ &0.25 \cdot \text{clearcoat} \cdot f_{\text{clearcoat}} + \\ &(1 - \text{metallic}) \cdot \text{specTrans} \cdot f_{\text{glass}} \end{aligned}$$

Glass reflection is not explicitly included; its energy is added via $\hat{f}_{\text{metal}}$.

3.4 Ambient Occlusion

To approximate the effects of global illumination in a computationally efficient manner, our material system incorporates an ambient occlusion (AO) texture. The AO map encodes the degree of occlusion at each surface point, simulating the energy loss caused by multiple light bounces being absorbed in concave areas, such as creases, corners, and cavities.

During indirect lighting computation, the AO texture is sampled and used as a scalar factor that modulates the intensity of the indirect light contribution. This helps to darken areas where less light is expected to reach due to geometric self-occlusion, thereby enhancing the perceived realism of the shading.

4 Results

After we implemented our Monte Carlo Path Tracing rendering pipeline, we render the scene in two methods: Lightweight BSDF based on Methodology 3.2, and Disney BSDF using techniques in Methodology 3.3. Each result section contains an outcome image and unresolved limitations. Finally, we concluded challenges resolved during project development.

4.1 Lightweight BSDF

Shown as Figure 4, renderer with lightweight BSDF generates an image with exquisite details and realistic lighting. The renderer successfully performed: specular reflection with diffusion on left wall in pink, wooden table, and IMAC screen; complete specular reflection on right-hand side mirror; refraction on glass window, rabbit, and guitar. With global illumination and area lights from ceiling and the table lamp, we observed realistic soft shadows from table, IMAC, anime character, and mirror. In terms of effiency, Lightweight BSDF pipeline generates outputs around 2 hours; comparing to rendering pipeline only using Monte Carlo ray tracing (Figure 5), our pipeline has a higher effiency and quality.

4.1.1 Lightweight BSDF Limitations

Light from the table lamp was weaker than our expectation; we expected to observe highlight areas on the wooden table and wall (comparing lighting in Figure 5). We believe that this was caused by the Next Event Estimation algorithm (NEE). In NEE, the algorithm samples a random surface position among all area lights based on their surface area, and returns reward based on ray from surface position to ray tracing interaction. However, the light bulb in table lamp is a small sphere; it has lower surface area compared with ceiling light, and half of the NEE sampling ray will be blocked

Figure 4: Image Rendered Using Lightweight BSDF
size: 1365 x 1024 pixels | spp: 256 | average max depth: 10 | render time: 2h

Figure 5: Comparison between Image Renered by Monte Carlo (left) and Lightweight BSDF (right)
Image by Monte Carlo size: 512 x 512 pixels | average max depth: 1000 | spp: 4096 | render time: 12h

Figure 6: Disney BSDF rendering result
size: 1344 x 768 pixels | spp: 256 | render time: 30min

by itself – causing biased sampling. We could address this issue by raising weight of lighting for table lamp, but as a result, this leads to higher sampling variance – one of main causes to fireflies (see 4.3) in output.

Shadows from transparent objects were stronger than expectation; we expected to observe little shadow from completely transparent objects (comparing shadow of transparent guitar in Figure 5). It was also caused by NEE algorithm. Currently, NEE returns reward without shadow test; as a result, it returns 0 even if the ray can hit the light after passing through some transparent objects and causes stronger shadow beneath transparent objects. If we enabled shadow test in NEE, the algorithm shall cover those rare cases, but meanwhile its sampling result would have a higher variance forcing us to use a higher SPP and Ray Trace Depth to converge.

4.2 Disney BSDF

Figure 6 shows the Disney BSDF rendering result, we incorporated a variety of materials, including metal, jade, glass, vegetation, wood, and fabric, to simulate how different surfaces interact with light under natural illumination. This extends the previous model—limited to random diffuse reflection and fixed specular reflection/refraction—into a broader and more unified material representation. We use Embree to speed up, spending about 30 minutes to rendering a 1344x768 resolution image with 256 spp.

4.2.1 Disney BSDF Limitations

The current subsurface scattering effect is weak, making the jade material appear insufficiently translucent. This is primarily due to the 2012 Disney BRDF's linear blend of diffuse and subsurface terms, which are computed separately and lack physical coherence. The 2015 Disney BSDF improves this by introducing a physically-based diffusion profile:

$$R_d(r) = \frac{e^{-r/d} + e^{-r/3d}}{8\pi dr},$$

where r is the radial surface distance and d is the effective diffusion length. This model produces smoother and more realistic subsurface light transport, better suited for materials like jade.

4.3 Challenges Resolved During Development

Low Indirect Lighting in Open Scene Monte Carlo path tracing relies on global illumination to generate both direct and indirect lighting. In an open scene, however, much of the indirect light simply gets wasted into empty space rather than illumining other objects. As a result, those objects receive less lighting causing some areas to appear unnaturally dark. Our solution is sealing the scene into a box to enhance indirect lighting.
Moreover, different from rasterization, path tracing must simulate each light bounce explicitly. In practice, this caused challenges on controlling overall brightness: increasing intensity of a point light can create bright hotspots and leaving it too weak yields patchy shadows and dark corners. A practical solution is to enlarge emission surfaces, so that light is more evenly distributed comparing to enlarge emission strength.

Fireflies Debug During our project development, one of the most common errors on render result is "fireflies" – isolated, overly bright pixels that suggest non-conserved light energy or biased ray tracing. However, there are various causes to fireflies: low max depth and spp resulting insufficient sampling, incorrect material parameters leading to energy leaks, or wrong ray tracing implementation. In this case, we increased max depth and spp to remove most of the fireflies; if fireflies were still abundant, we had to systematically inspect our model and algorithm for debugging.

Non-Manifold Objects For non-manifold objects, when a ray hits their surfaces it can be difficult to determine whether the intersection is interior or exterior. To resolve this, we assume all non-manifold objects are completely opaque, so any intersection with them is treated as exterior.

Finetuning Material Parameters Fine-tuning material parameters for photorealistic rendering is difficult, because material parameters simulate complex surface behaviors with a few values—color, roughness, specular weight, IOR, etc. Small misalignments will make surfaces look "almost real" but just off enough to break immersion. Those near-real outcomes can result unsettling feelings, so achieving truly realistic materials requires time-consuming, iterative adjustments.

Error in Weighted Combination of the Unified Disney BSDF Model There was an error in the weighted combination of the unified Disney BSDF model. Specifically, the specular reflection contribution from the glass component was initially omitted during the final BSDF weighting computation. Instead, an incorrect expression $\texttt{metal} \cdot \hat{f}_{\text{metal}}$ was used to compute the metallic reflection. After consulting relevant references, this was corrected to:

$$(1 - \text{specTrans} \cdot (1 - \text{metallic})) \cdot \hat{f}_{\text{metal}}$$

This revised formulation accounts for both the specular reflection of non-metallic materials and the metallic reflection in a unified manner.

Incorrect Normal Direction When Using Normal Maps Dark edges appeared after introducing normal maps due to incorrect lighting caused by mismatched normals. In areas with rapid surface orientation changes, the normal map may produce normals opposite to the geometric normal, resulting in negative dot products with the light direction and visual artifacts. To resolve this, we fallback to the geometric normal when the dot product between the two normals is negative, ensuring consistent shading.

5 Conclusion

Comparing to the inspiration image, we achieved improvements on scene building, variety on reflection effects, character modeling and rendering techniques (Disney BSDF). In this project, our render pipeline utilized the lightweight BSDF and Disney BSDF to achieve path tracing. Throughout development, we recognized limitations of those techniques and resolved challenges to obtain better outcome images. In our work in methodology and findings in result, we noticed several cost-performance tradeoffs – computational cost or complexity of material parameter vs photorealistic result, and this report would be helpful to further study on computer graphic.

As future work, this project needs improvement on ray tracing sampling algorithm (e.g. cosine weighted hemisphere sampling), computational efficiency (e.g. GPU compatibility), and modeling compatibility (e.g. supporting non-triangle mesh, character skeleton).